

Compiling and Running Packages

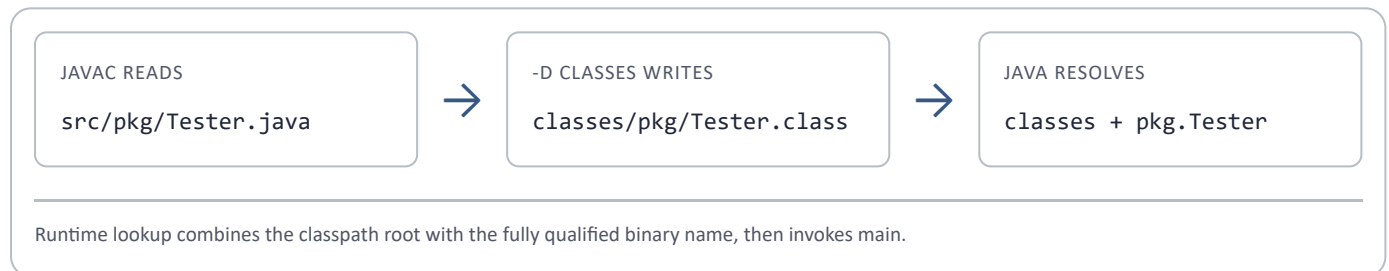
Compile source into package directories, then run the fully qualified class from the correct classpath root.

This complete offline chapter connects ideas, diagrams, exact traces, commented Java and exam practice. Switch themes for reading; printing always uses the light presentation and includes every required trace and answer.

GUIDE 2026-10-07.1 · STANDALONE HTML AND PDF SHARE THE SAME CONTENT

1. The problem and the plain-language idea

Compile source into package directories, then run the fully qualified class from the correct classpath root.



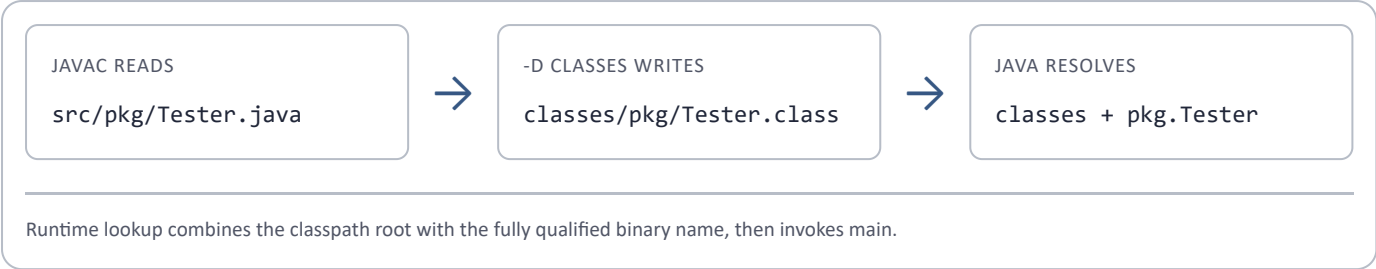
Think of a package as a class address. Two classes may share a simple name when their package addresses differ. Source paths and compiled paths are measured from different roots.

Source root: src/	Output / classpath root: classes/
src/	classes/
pkg/	pkg/
A.java	A.class
B.java	B.class
Tester.java	Tester.class
app/	app/
Main.java	Main.class

Package identity is below each root. src and classes are roots, not parts of the package name.

2. Definition and essential vocabulary

Compilation translates Java source into class files. Running a packaged class combines a classpath root with a fully qualified class name and invokes its main method.



Term	Meaning in this chapter
javac	Java compiler: source .java to compiled .class.
-sourcepath src	Where to find source dependencies.
-d classes	Where to write compiled package directories.
-classpath classes	Where Java begins lookup of compiled classes.
Fully qualified name	pkg.Tester, not a slash path and not a filename.



Source root: src/	Output / classpath root: classes/
src/ pkg/ A.java B.java Tester.java app/ Main.java	classes/ pkg/ A.class B.class Tester.class app/ Main.class
Package identity is below each root. src and classes are roots, not parts of the package name.	

3. Visual workflow: declare, compile, run

Follow the same class through three representations

▼ Full trace table - select a row to jump to its scene

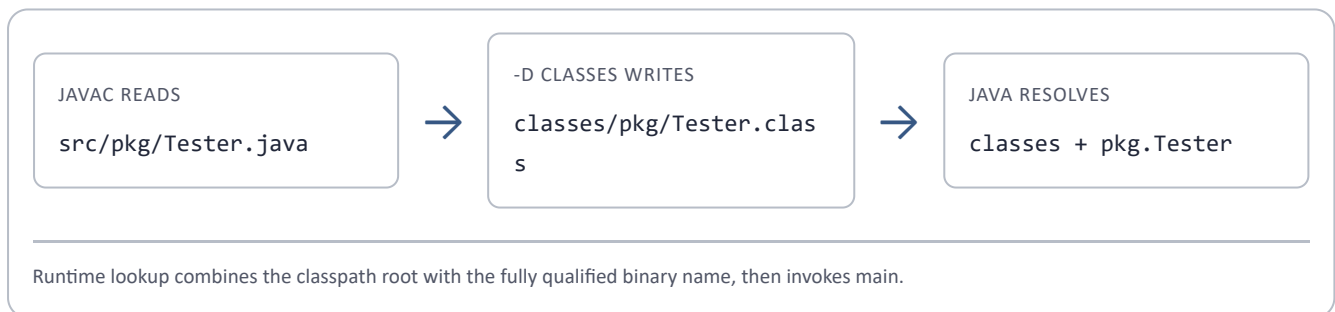
Step	Stage	Input	Output / resolution
1	Declare	package pkg; / src/pkg/A.java	pkg.A
2	Compile	javac -d classes -sourcepath src ...	classes/pkg/*.class
3	Run	java -classpath classes pkg.Tester	classes/pkg/Tester.class → main

1. Declare identity



package pkg makes the public class A identifiable as pkg.A. The file belongs below source root src in pkg/A.java.

3. Resolve and run



java -classpath classes pkg.Tester locates classes/pkg/Tester.class and invokes its main method.

4. Rules before code

- Put a package declaration before import declarations and the class declaration.
- Match the package path below the source root.
- Match a public class name to its source filename, including case.
- Classes in the same package do not need imports for one another.
- Named packages cannot import a class from the unnamed default package.

- An import does not change an access modifier.
- Use a fully qualified class name when launching a packaged program.

Representation	Example	Root
Package	pkg	Not a filesystem root
Source	src/pkg/A.java	src
Compiled	classes/pkg/A.class	classes
Binary name	pkg.A	Root supplied separately by classpath

5. Fully commented Java and commands

JAVA

```
package pkg; // Identity pkg.A; file src/pkg/A.java.

public class A { // Public class name must match A.java.
    // Other packages may call this public method on this public class.
    public static void greet() {
        System.out.println("Hello from pkg.A");
    }
}
```

JAVA

```
package pkg; // File src/pkg/B.java.

public class B {
    public static int twice(int x) { return 2 * x; }
    // No access modifier: only classes in package pkg may call secret.
    static String secret() { return "pkg only"; }
}
```

JAVA

```
package pkg; // File src/pkg/Tester.java.

public class Tester {
    // This exact main signature supplies the program entry point.
    public static void main(String[] args) {
        A.greet(); // Same package: no import required.
        System.out.println(B.twice(21)); // Prints 42.
        System.out.println(B.secret()); // Legal: Tester and B share pkg.
    }
}
```

JAVA

```
package app; // File src/app/Main.java.
```

```
import pkg.A; // Import a public class from another package.
import pkg.B;

public class Main {
    public static void main(String[] args) {
        A.greet();
        System.out.println(B.twice(5)); // Prints 10.
        // B.secret(); // Would fail: imports do not grant package-private access.
    }
}
```

COMMANDS

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java src/pkg/Tester.java src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

6. Visibility: imports and access are different

Member modifier	Same class	Same package	Subclass outside package	Unrelated outside class
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes, with subclass access restrictions	No
none (package-private)	Yes	Yes	No	No
private	Yes	No	No	No

Protected access from a different package is through inheritance and is subject to receiver restrictions; it is not blanket permission to access any superclass object. A top-level class may be public or package-private, not protected/private.

ACCESS COMPARISON

```
import pkg.B;
// Legal in app.Main: B.twice(5)
// Illegal in app.Main: B.secret()
// Importing B does not alter secret() visibility.
```

7. Source-to-output trace table

Stage	Given	Resolution	Result
Declare	package pkg; / Tester.java	pkg.Tester	src/pkg/Tester.java

Stage	Given	Resolution	Result
Compile	javac -d classes -sourcepath src ...	Write below output root	classes/pkg/Tester.class
Run	java -classpath classes pkg.Tester	Find class and invoke main	Hello from pkg.A / 42 / pkg only
Cross-package	java -classpath classes app.Main	Public APIs of pkg.A and pkg.B	Hello from pkg.A / 10

PKG.TESTER OUTPUT

Hello from pkg.A
 42
 pkg only

APP.MAIN OUTPUT

Hello from pkg.A
 10

8. Five worked cases

1. One-level package

GIVEN
 package pkg; public class A

→

RESOLUTION / RESULT
 src/pkg/A.java → classes/pkg/A.class

Identity is pkg.A.

Step	Evidence	Conclusion
1	package pkg; public class A	Identify package/type/root
2	src/pkg/A.java → classes/pkg/A.class	Identity is pkg.A.

2. Nested package

GIVEN
package ub.cs.tools; public class Helper

→

RESOLUTION / RESULT
src/ub/cs/tools/Helper.java →
classes/ub/cs/tools/Helper.class

Identity is ub.cs.tools.Helper; roots are excluded.

Step	Evidence	Conclusion
1	package ub.cs.tools; public class Helper	Identify package/type/root
2	src/ub/cs/tools/Helper.java → classes/ub/cs/tools/Helper.class	Identity is ub.cs.tools.Helper; roots are excluded.

3. Same-package call

GIVEN
pkg.Tester calls B.secret()

→

RESOLUTION / RESULT
No import; package-private access allowed.

Both classes belong to pkg.

Step	Evidence	Conclusion
1	pkg.Tester calls B.secret()	Identify package/type/root
2	No import; package-private access allowed.	Both classes belong to pkg.

4. Cross-package call

GIVEN
app.Main imports pkg.B

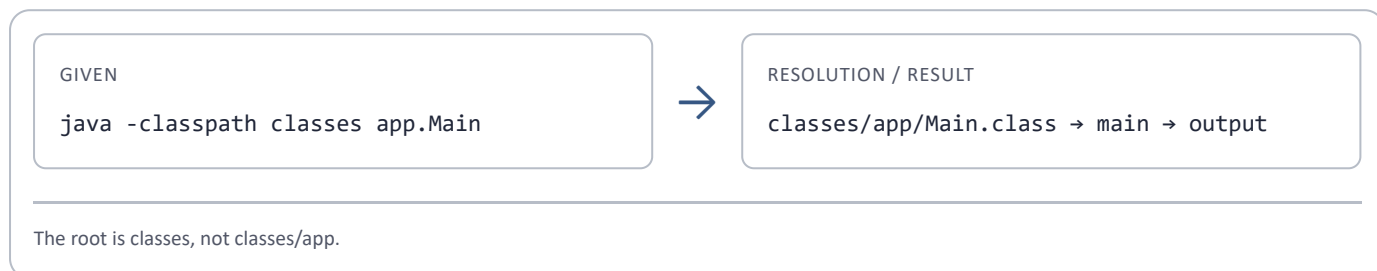
→

RESOLUTION / RESULT
B.twice(5) → 10; B.secret() is
inaccessible.

Import shortens the class name but does not grant member access.

Step	Evidence	Conclusion
1	app.Main imports pkg.B	Identify package/type/root
2	B.twice(5) → 10; B.secret() is inaccessible.	Import shortens the class name but does not grant member access.

5. Runtime root



Step	Evidence	Conclusion
1	java -classpath classes app.Main	Identify package/type/root
2	classes/app/Main.class → main → output	The root is classes, not classes/app.

9. Build or explain it from scratch

- Choose the package and public class names.
- Create matching source directories.
- Write the package declaration first.
- Add a main entry point to the class you intend to run.
- Compile with an output root and source dependency root.
- Run with a classpath root and fully qualified name.
- Check output and exercise cross-package public access.

10. Properties and when to use packages

Packages organize names and collaboration; imports are compile-time name resolution conveniences. A wildcard is not a performance optimization. There is no useful sorting-style $O(n)$ claim to assign to an import declaration.

Property	Practical consequence
Namespace	Same simple type names may exist in different packages.

Property	Practical consequence
Encapsulation	Package-private members form an internal package API.
Maintainability	Related classes have predictable source locations.
Dependency lookup	Sourcepath/classpath roots must locate the required types.
Case sensitivity	pkg.A and pkg.a name different types.

11. Common errors and exact repairs

Message / symptom	Likely cause	Repair
cannot find symbol: Scanner	Simple name is not in scope.	Add import java.util.Scanner or use the fully qualified name.
reference to Date is ambiguous	Two wildcard-imported packages contain Date.	Use a specific import or fully qualified name.
package pkg does not exist	Compiler cannot locate sources/compiled dependencies.	Use -sourcepath src or -classpath classes as appropriate.
class A is public, should be declared in A.java	Public class and filename disagree.	Use A.java with public class A.
Could not find or load main class pkg.Tester	Wrong classpath root/name or missing compiled file.	Use classes as root and pkg.Tester without .class.
Main method not found in class pkg.A	That helper is not an entry-point class.	Run pkg.Tester or app.Main.
secret() is not public in B	Cross-package access to a package-private member.	Use a public API; an import cannot grant visibility.

12. Exam-style practice and answers

A. Place Helper in ub.cs.tools. Give declaration, source path, output path and binary name. [4]

package ub.cs.tools; / src/ub/cs/tools/Helper.java / classes/ub/cs/tools/Helper.class / ub.cs.tools.Helper. The src and classes roots are not package segments.

B. Explain import java.util.* and whether it includes subpackages. [3]

It allows accessible types directly in java.util to be named simply. It does not import types from subpackages, copy code or install dependencies.

C. Why can pkg.Tester call B.secret while app.Main cannot? [3]

secret has no access modifier, so it is package-private. Tester shares pkg. Main belongs to app; importing B does not grant member visibility.

D. Repair java -classpath classes/pkg Tester.class. [3]

Use java -classpath classes pkg.Tester. The classpath names the root above package folders; the launch argument is a dotted binary name without an extension.

E. Compile and run the four classes in this chapter. State expected output. [6]

COMMANDS

```
# Run from the project folder that contains src and classes.
javac -d classes -sourcepath src src/pkg/A.java src/pkg/B.java src/pkg/Tester.java src/app/Main.java

# Classpath points to the ROOT above pkg and app, not to classes/pkg.
java -classpath classes pkg.Tester
java -classpath classes app.Main
```

pkg.Tester prints Hello from pkg.A, 42, pkg only. app.Main prints Hello from pkg.A, 10.

F. Scanner is unresolved even though the JDK is installed. What are two fixes? [2]

Add import java.util.Scanner; or use java.util.Scanner at every relevant type/constructor use. Both name the same class.

13. Test-ready summary and mastery check

Compile source into package directories, then run the fully qualified class from the correct classpath root.

- I can separate package declaration, import and access modifier.
- I can translate dots to directories below a specified root.
- I can explain specific/wildcard/fully qualified alternatives.
- I can identify the main class and run it without an extension.
- I can diagnose naming, lookup and visibility errors separately.

Structural reference: the author-supplied CSI247 guide, with topic-specific adaptation and repo-owned teaching content. All traces in this artifact are generated by deterministic simulations of its stated procedures. Counts distinguish key comparisons from loop tests, probes, swaps, shifts and array writes.